

Introduction to Rust for Java Developers



Eugenio Alvarez

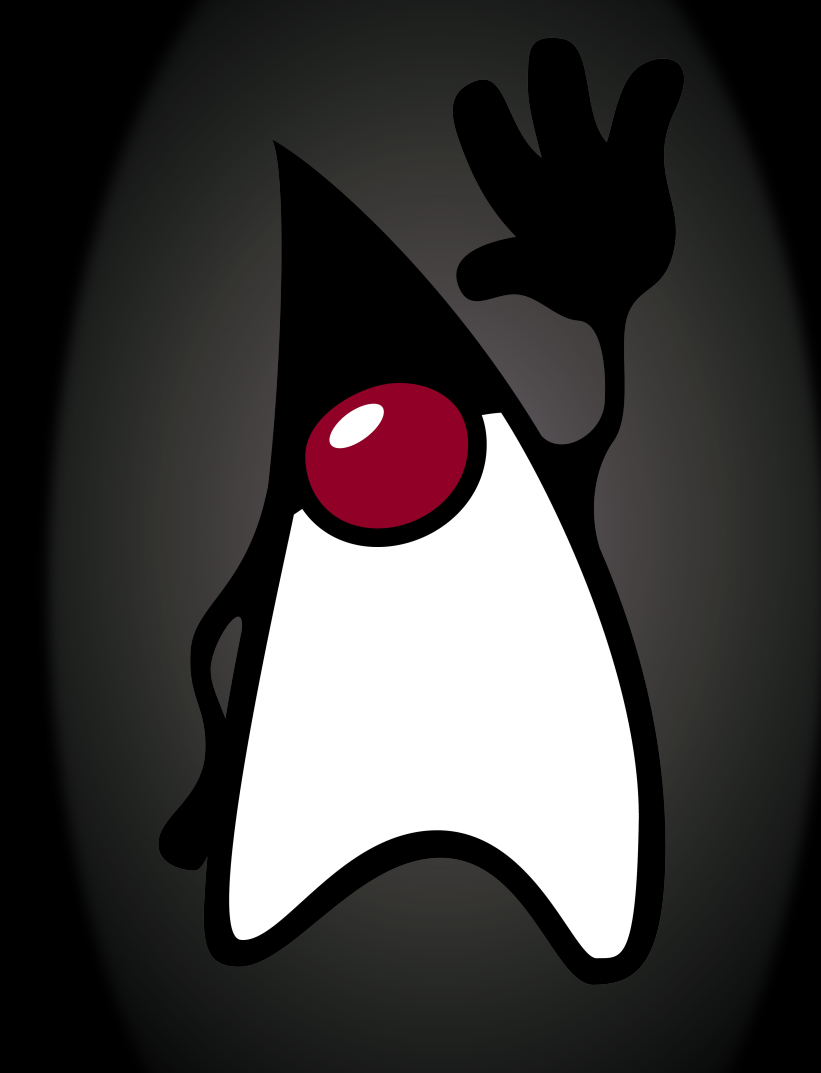


Thursday, December 5th 2024 @ 6:30PM EST  **zoom**

Miami JVM Group (MJUG)

<https://www.meetup.com/miami-java-user-group/>

EUGENIO ALVAREZ



A South Florida software engineering professional. Experienced in organizational design, software design, construction, and deployment. Extensive knowledge of Java. Proponent of Unit testing. An advocate for Agile Software Engineering methods using Kanban and Scrum.

www.linkedin.com/in/ealvarez



WHAT IS THIS VIDEO ABOUT

- The Rust programming language has consistently topped Stack Overflow's annual Developer Survey as the "most-admired programming language." We will discuss why Rust is so admired, compare it to Java, and investigate Rust use cases to understand how they relate.
- The 2024 Stack Overflow Survey Results show that Rust is the most admired programming language, making it the most loved language since 2016.

Source: 2024 Stack Overflow Developer Survey: <https://survey.stackoverflow.co/2024/technology#admired-and-desired>

BRIEF RUST HISTORY

WHERE DID RUST COME FROM?

- Created by Graydon Hoare as a personal project while working at Mozilla Research
- Hoare has stated that Rust was named for the group of fungi that are "over-engineered for survival"^[1]

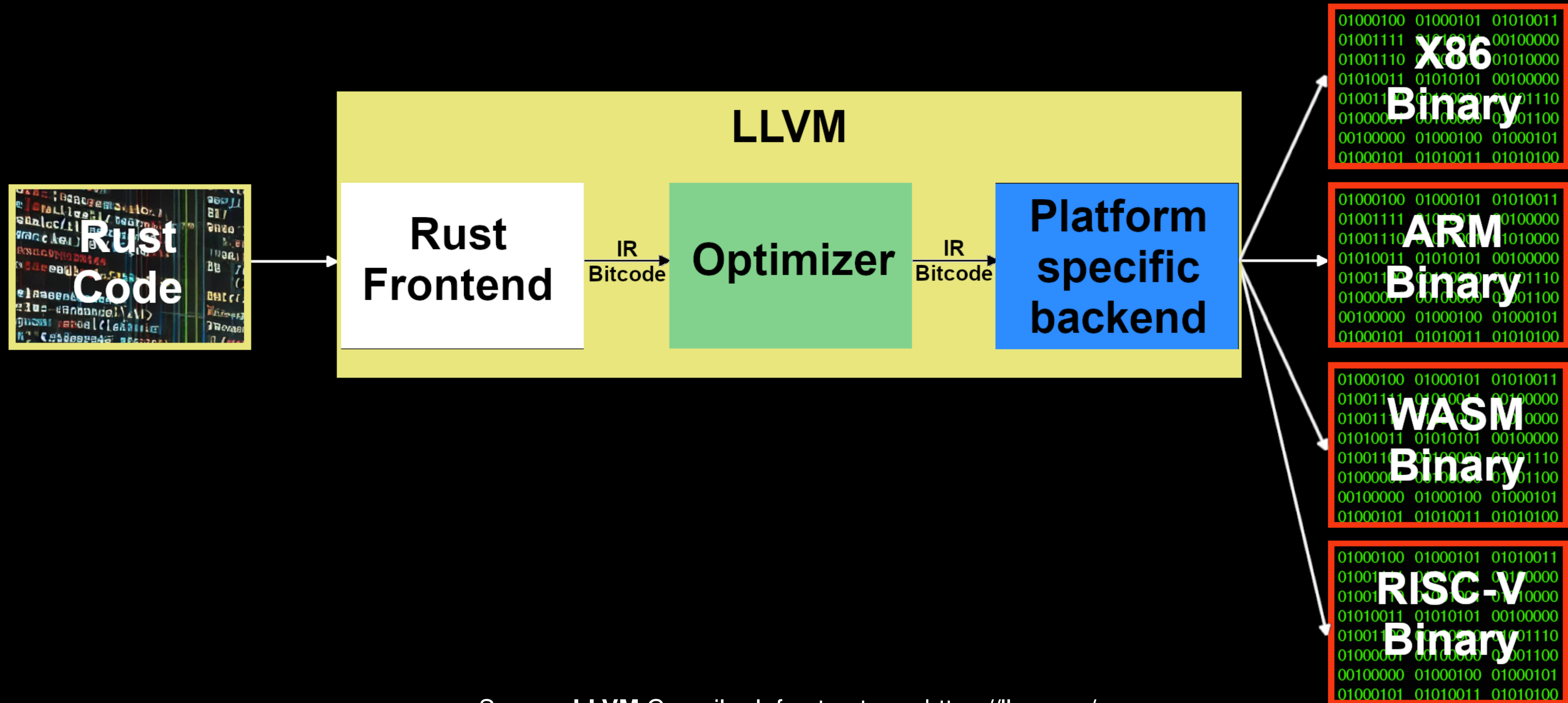


moz://a

Source: [1]: <https://www.technologyreview.com/2023/02/14/1067869/rust-worlds-fastest-growing-programming-language/>



RUST COMPILER INFRASTRUCTURE



Source: LLVM Compiler Infrastructure : <https://llvm.org/>

Source: Rustc Platform Support: <https://doc.rust-lang.org/rustc/platform-support.html>



WHY IS RUST SO ADMIRERD?



WHY ALL THE LOVE

- Rust is generally considered the first widely adopted language to achieve memory safety without relying on a garbage collector with comparable performance and memory usage to C and C++.
- How is memory safety achieved without garbage collection?
 - Rust's ownership system and borrow checker
 - Checks at compile time: "If it compiles, it works!"

Source: 2024 Stack Overflow Developer Survey: <https://survey.stackoverflow.co/2024/technology#admired-and-desired>



WHY ALL THE LOVE (PARTIE DEUX)

- High-performance concurrency, aka “Concurrency without fear” is achieved with the ownership model and borrow checker at compile time.
- Zero-cost abstraction language features enable high-level code without incurring runtime overhead.
- Developer centered design like compiler error messages

Source: 2024 Stack Overflow Developer Survey: <https://survey.stackoverflow.co/2024/technology#admired-and-desired>



CLOUD PROVIDERS LOVE RUST

- A 2017 study of Rust-based code found Rust uses half as much electricity as a similar program written in Java.
- “Rust is the fastest growing language at . It's had such an impact that we've actually rewritten a lot of our critical code in Rust” Chris Betz, AWS CISO June 2024.

Source: Energy Efficiency Across Programming Languages: <https://sites.google.com/view/energy-efficiency-languages/>

Source: AWS re:Inforce 2024 - Keynote with Chris Betz: <https://www.youtube.com/watch?v=skH3Q90llss>



OPERATING SYSTEM VENDORS LOVE RUST

- Microsoft is rewriting parts of the Windows 11 kernel using the memory-safe programming language Rust (May 2023)



- Linux kernel version 6.1 (2022) introduced initial support for Rust

Source: First Rust Code in the Windows 11 Kernel: <https://www.thurrott.com/windows/windows-11/282995/first-rust-code-shows-up-in-the-windows-11-kernel>

Source: Rust for Linux: https://en.wikipedia.org/wiki/Rust_for_Linux



THE UNITED STATES GOVERNMENT LOVES RUST AND JAVA TOO

- Darpa's TRACTOR Project (Translating C to Rust)
- The Case for Memory Safe Roadmaps (CISA, NSA, FBI etc.)
 - C#
 - Python
 - Go
 - Rust 🦀
 - Java
 - Swift

Source: In Rust we trust? White House Office urges memory safety:

<https://stackoverflow.blog/2024/03/04/in-rust-we-trust-white-house-office-urges-memory-safety/>

Source: The Case for Memory Safe Roadmaps:

<https://media.defense.gov/2023/Dec/06/2003352724/-1/-1/0/The-Case-for-Memory-Safe-Roadmaps-TLP-clear.pdf>



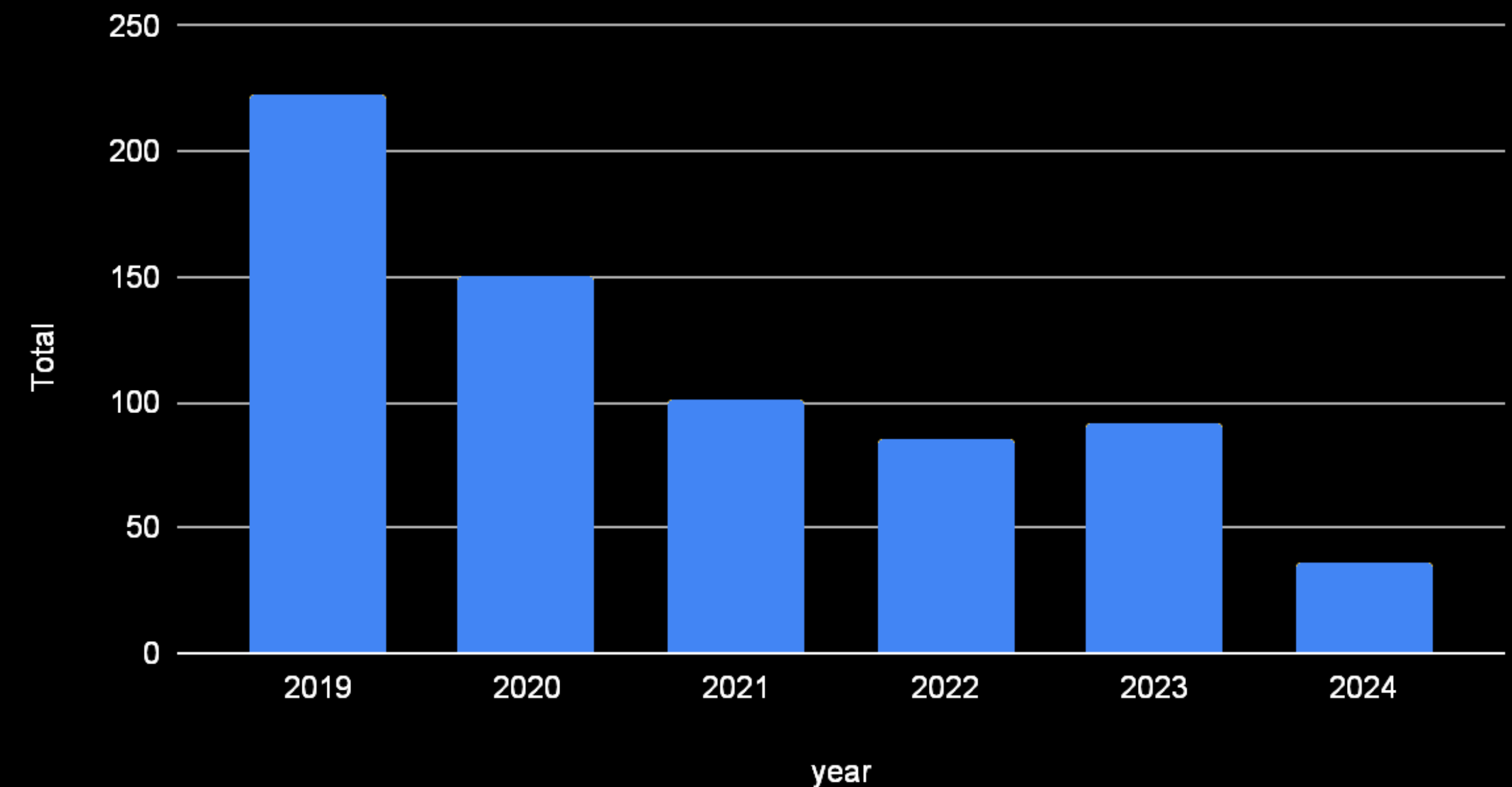
OPERATING SYSTEM VENDORS LOVE RUST

- “The percentage of memory safety vulnerabilities in Android dropped from 76% to 24% over 6 years as development shifted to memory safe languages” (September 2024)

Total Memory safe and Memory Unsafe Lines of Code in AOSP



Number of Memory Safety Vulns per Year



Source: Eliminating Memory Safety Vulnerabilities at the Source

<https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html>



C/C++ REACTION TO RUST

- Safe C++ proposal
 - <https://safecpp.org/draft.html>
- Fil-C
 - <https://github.com/pizlonator/llvm-project-deluge/blob/deluge/Manifesto.md>
- TrapC
 - <https://vimeo.com/1028578347>
- Enabling hardened libc++ / listedc++
 - <https://security.googleblog.com/2024/11/retrofitting-spatial-safety-to-hundreds.html>



RUST / JAVA ECOSYSTEMS

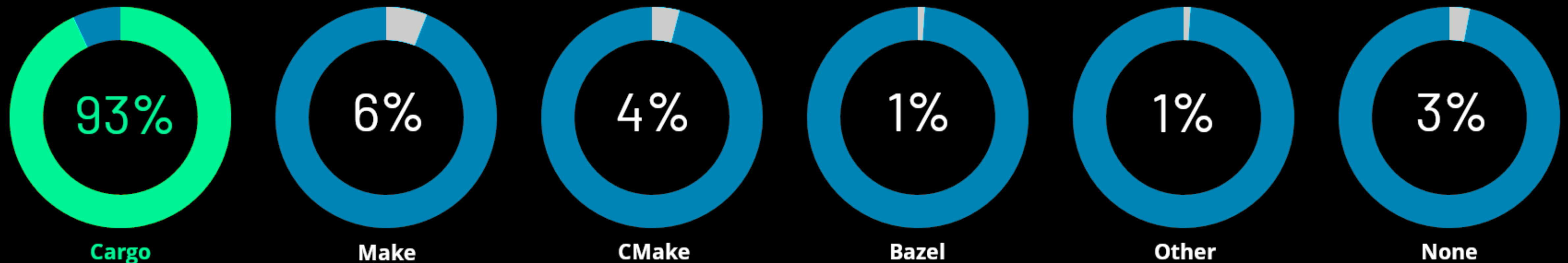


RUST ECOSYSTEM

- rustup (install/update)
- Clippy: linting tool (700 rules)
- Rustfmt: code formatter / checker
- Cargo: build / package manager

BUILDING RUST APPLICATIONS

Which build systems do you use to compile Rust?



Cargo • Make • CMake • Bazel

Source: JetBrains State of Developer Ecosystem 2023 : <https://www.jetbrains.com/lp/devecosystem-2023/rust/>



RUST CRATES

The screenshot shows the crates.io website. At the top, there's a navigation bar with the crates.io logo, a "Browse All Crates" link, and a "Log in with GitHub" button. Below this is a large green banner with the text "The Rust community's crate registry" and a search bar. Under the banner are two buttons: "Install Cargo" and "Getting Started". The main content area features statistics: "96,766,669,570 Downloads" and "164,076 Crates in stock". Below the statistics are three columns of featured crates: "New Crates" (imgfx v0.1.0, mcutie v0.1.0), "Most Downloaded" (syn, bitflags), and "Just Updated" (dutils v0.1.4, mm2 v0.13.1).

crates.io

Browse All Crates | Log in with GitHub

The Rust community's crate registry

Type 'S' or '/' to search

Install Cargo | Getting Started

Instantly publish your crates and install them. Use the API to interact and find out more information about available crates. Become a contributor and enhance the site with your work.

96,766,669,570
Downloads

164,076
Crates in stock

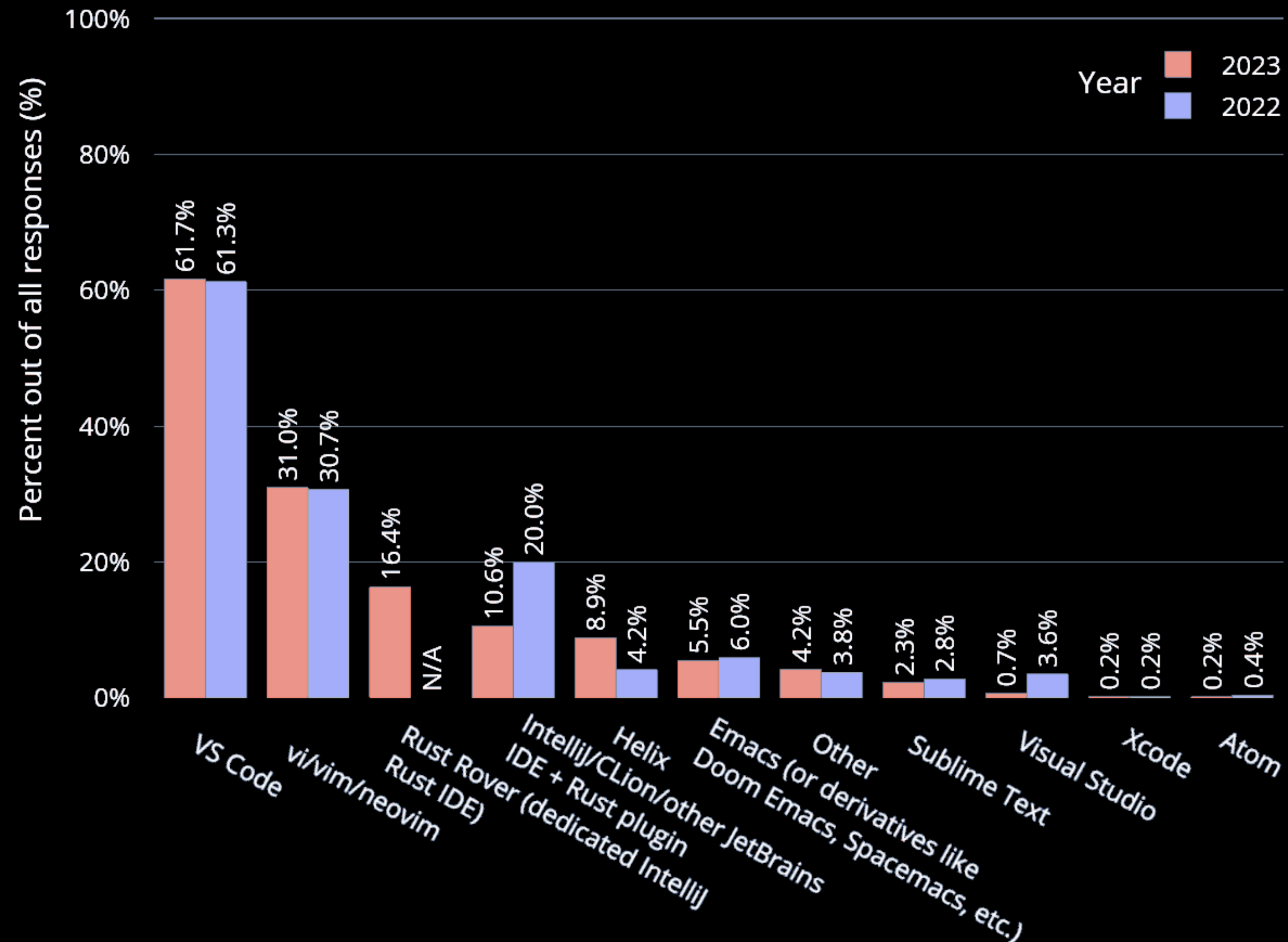
New Crates	Most Downloaded	Just Updated
imgfx v0.1.0	syn	dutils v0.1.4
mcutie v0.1.0	bitflags	mm2 v0.13.1

Source: The Rust community's crate registry : <https://crates.io/>

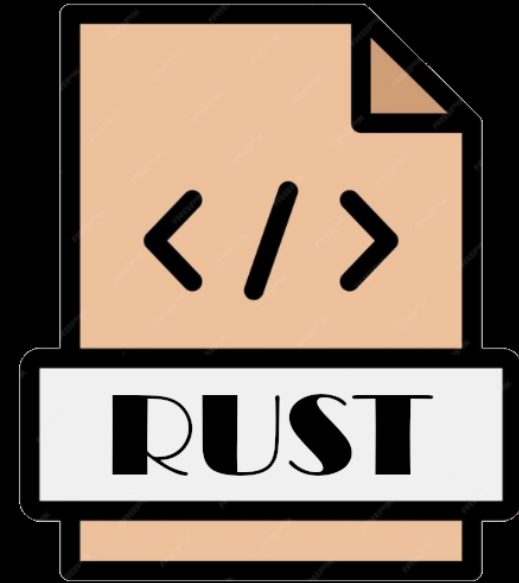
RUST IDE

Which editor or IDE setup do you use with Rust code on a regular basis?

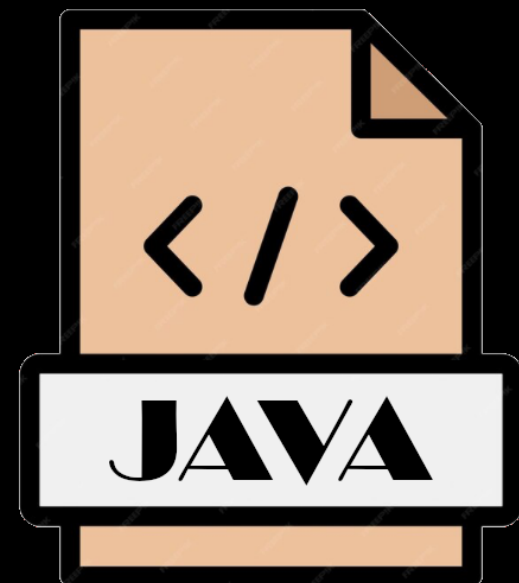
(total responses = 10480, multiple answers)



Source: 2023 Annual Rust Survey Results : <https://blog.rust-lang.org/2024/02/19/2023-Rust-Annual-Survey-2023-results.html>



RUST / JAVA LANGUAGE FEATURES





JAVA FEATURES NOT RUST



- Garbage Collector
- Inheritance
- Exceptions
- Reflection
- Null references

A STRING IN RUST COMPARED TO JAVA

- Strings are encoded in UTF-8
- **String** when you need an owned, mutable string.
- **&str** for borrowing or when working with literals.
- **CString** for C interoperability.
- **OsString** and **OsStr** for platform-specific needs.
- **Cow<str>** for efficient shared and owned string scenarios.

RUST TRAITS / JAVA INTERFACE

```
trait Animal {  
    fn speak(&self);  
}  
  
struct Dog;  
  
impl Animal for Dog {  
    fn speak(&self) {  
        println!("Woof!");  
    }  
}
```

```
interface Animal {  
    void speak();  
}  
  
class Dog implements Animal {  
    public void speak() {  
        System.out.println("Woof!");  
    }  
}
```

RUST OWNERSHIP

Example

```
fn main() {  
    let s1 = String::from("Hello, Rust!"); // 's1' owns the string.  
    let s2 = s1; // Ownership is moved to 's2'.  
  
    // println!("{}", s1); // ERROR: `s1` is no longer valid after the move.  
    println!("{}", s2); // Only 's2' can be used now.  
}
```

TESTING IN RUST

```
pub fn add(a: i32, b: i32) -> i32 {  
    a + b  
}  
  
#[cfg(test)]  
mod tests {  
    use super::*; // Import from the parent module  
  
    #[test]  
    fn test_add() {  
        assert_eq!(add(2, 3), 5); // Check if 2 + 3 equals 5  
    }  
}
```

Command line: cargo test



A FEW DATA POINTS



Feature	Java	Rust
Usage	Web/E-Commerce/ Banking/Medical	Systems/Web/IoT/ BlockChain
Deployment	Bytecode/ Native (GraalVM)	Native/WASM
Memory	GC	Ownership Model
Paradigm	OOP/Functional/ Generic/Declarative	Functional/Generic/ Declarative

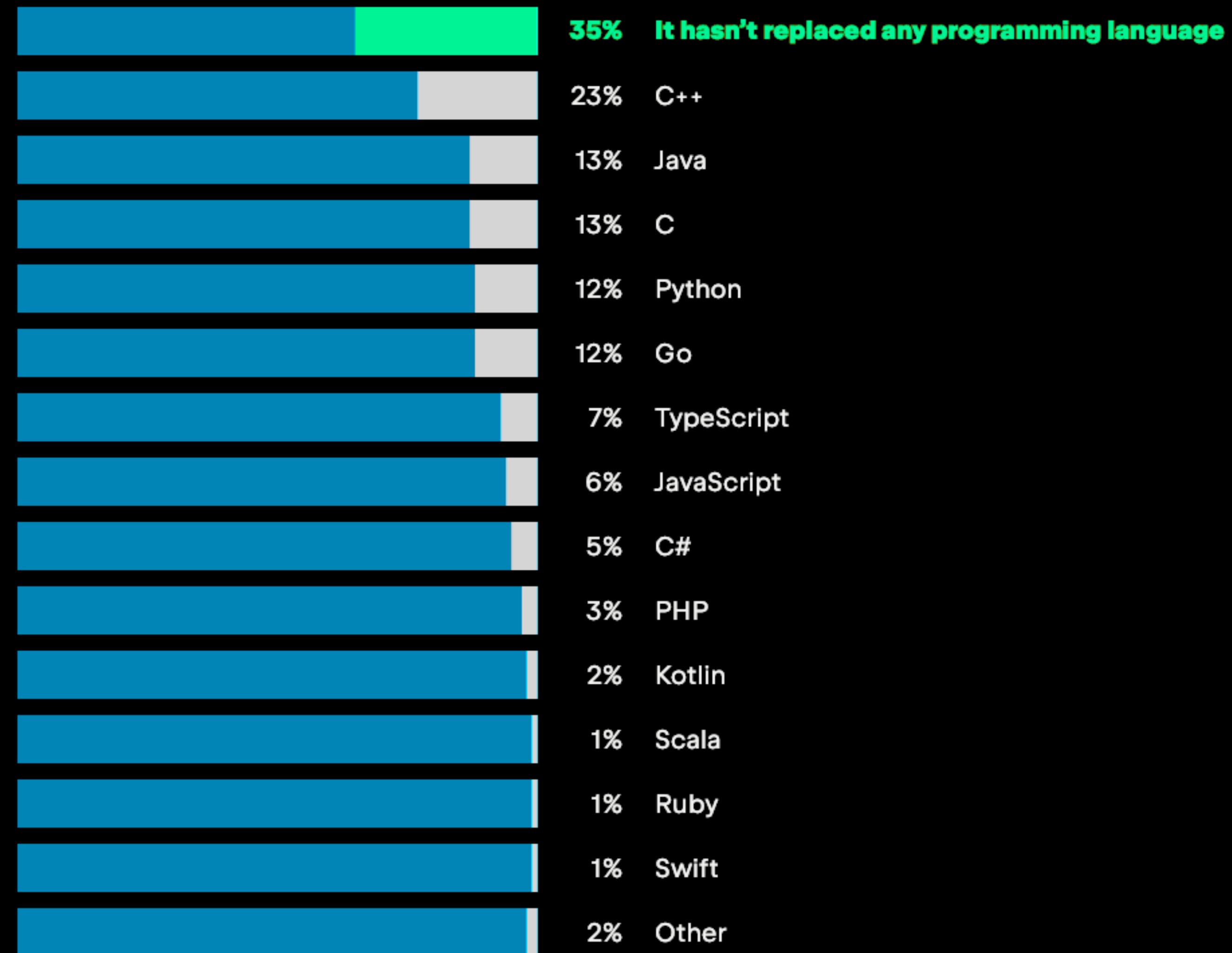


RUST USE CASES



RUST MIGRATION

From which programming language have you migrated to Rust?

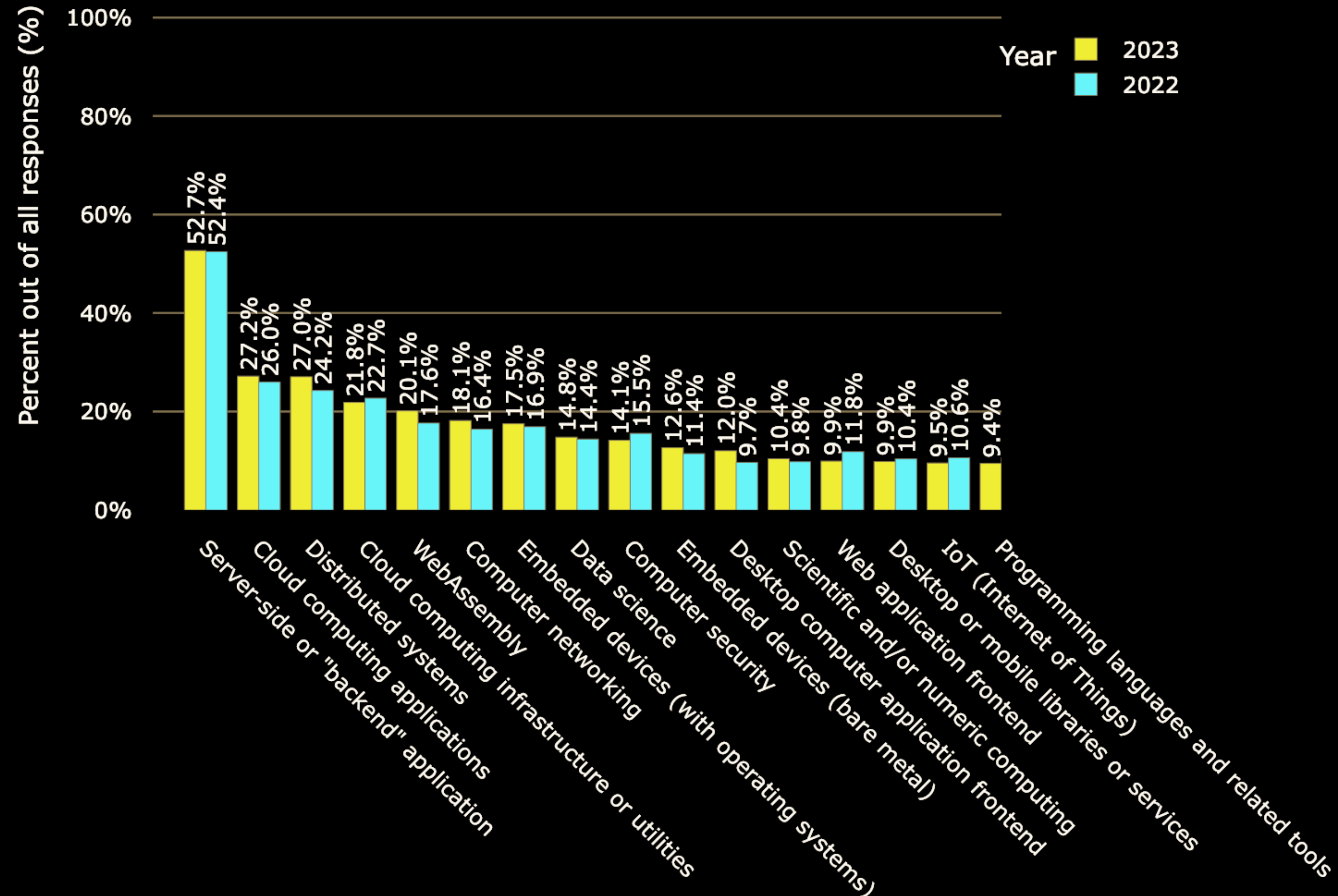


Source: JetBrains State of Developer Ecosystem 2023 : <https://www.jetbrains.com/lp/devecosystem-2023/rust/>

IN WHAT DOMAINS IS RUST BEING USED

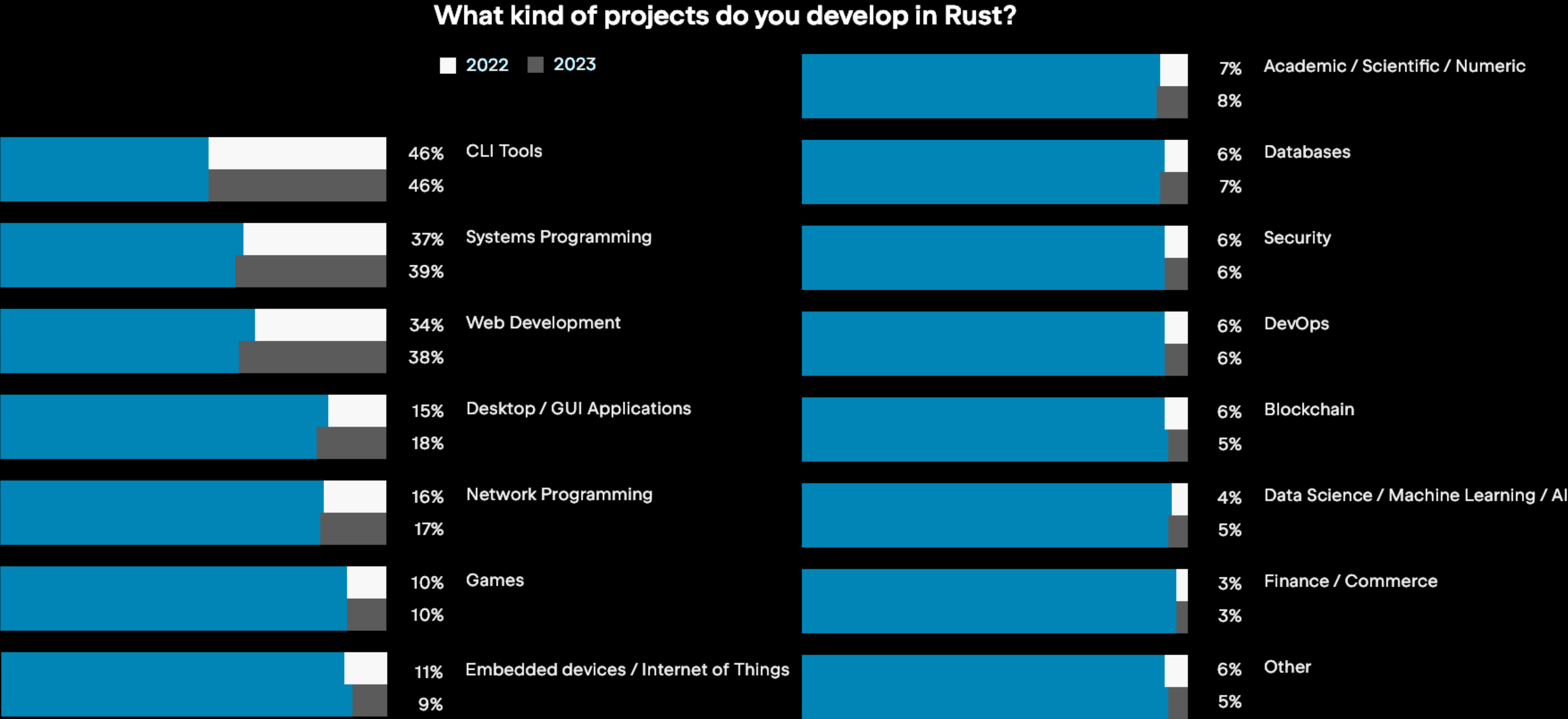
In what technology domain(s) is Rust used at your organisation?

(total responses = 4139, multiple answers)



Source: 2023 Annual Rust Survey Results : <https://blog.rust-lang.org/2024/02/19/2023-Rust-Annual-Survey-2023-results.html>

WHAT ARE DEVELOPERS MAKING WITH RUST?

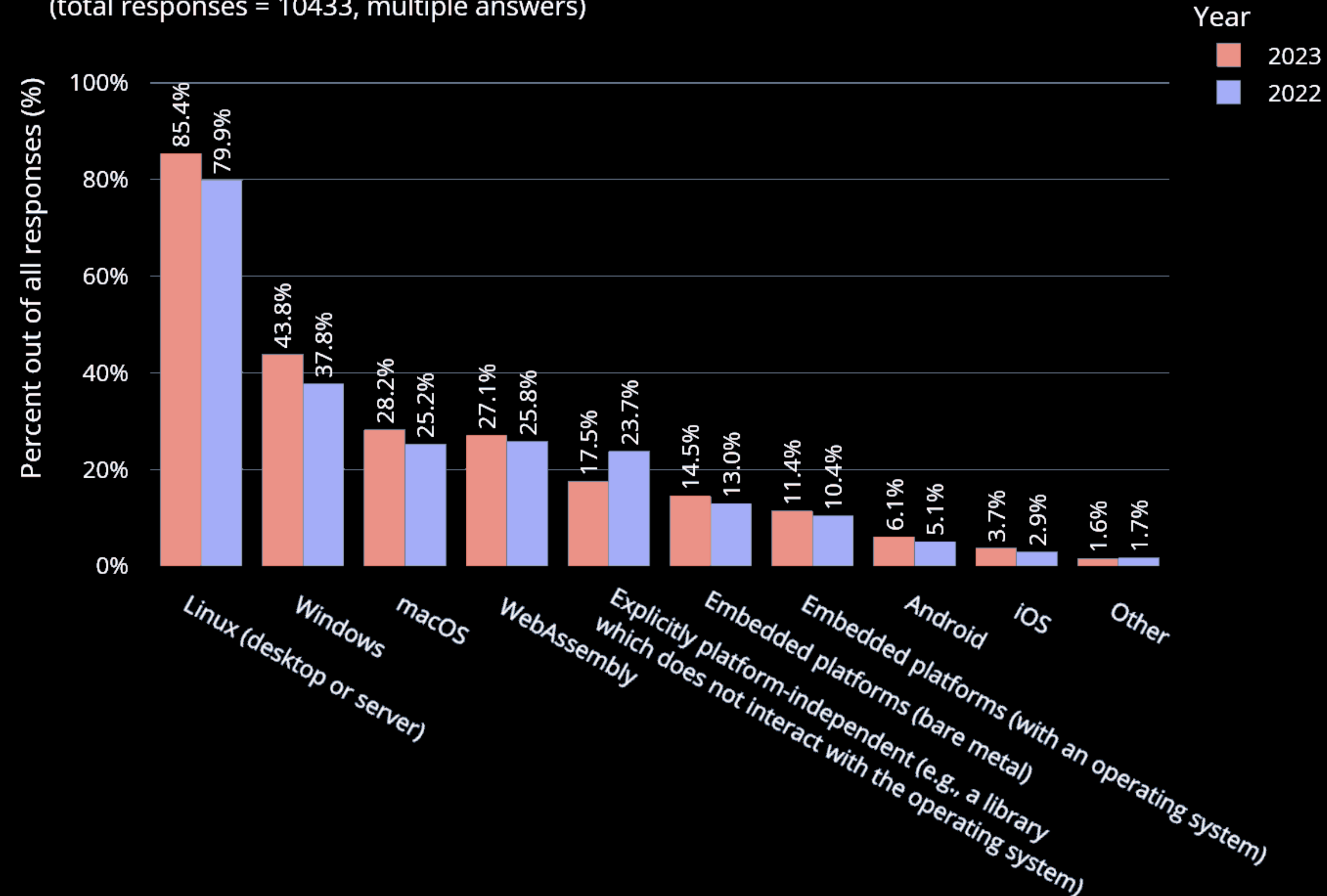


Source: JetBrains State of Developer Ecosystem 2023 : <https://www.jetbrains.com/lp/devecosystem-2023/rust/>

RUST TARGET OS/RUNTIMES

Which operating systems or runtimes do you develop Rust software for?

(total responses = 10433, multiple answers)



Source: 2023 Annual Rust Survey Results : <https://blog.rust-lang.org/2024/02/19/2023-Rust-Annual-Survey-2023-results.html>



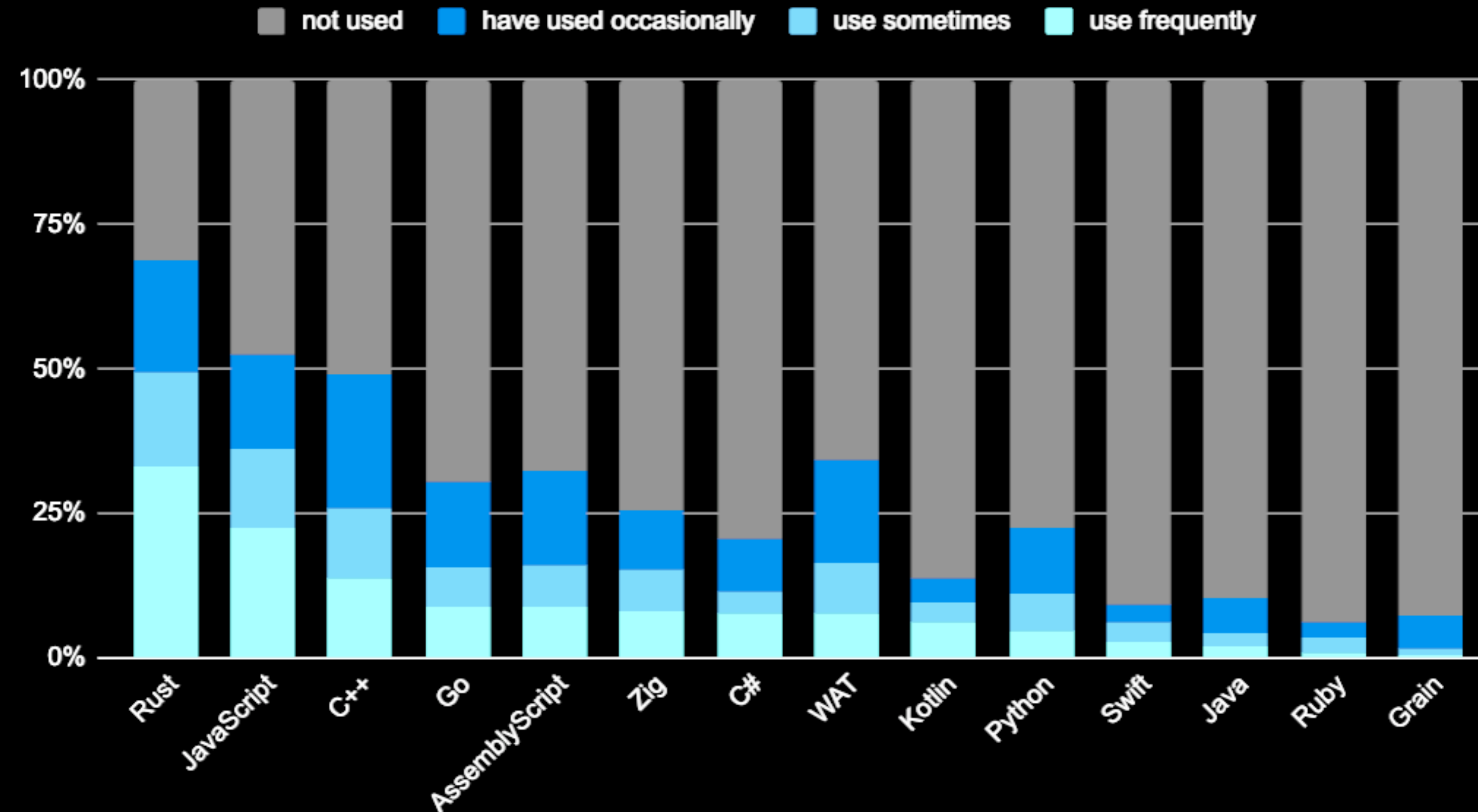
WEBASSEMBLY (WASM)

- Goal: Enable high-performance applications on web browsers.
- A binary instruction format for a stack-based virtual machine. Multiple languages can be used to create WASM, including 🦀 Rust.
- Supported by all major web browsers, including mobile web browsers for iOS and Android.



RUST AND WEBASSEMBLY (WASM)

Current WebAssembly language usage (n=303)



Source: The State of WebAssembly 2023: <https://blog.scottlogic.com/2023/10/18/the-state-of-webassembly-2023.html>

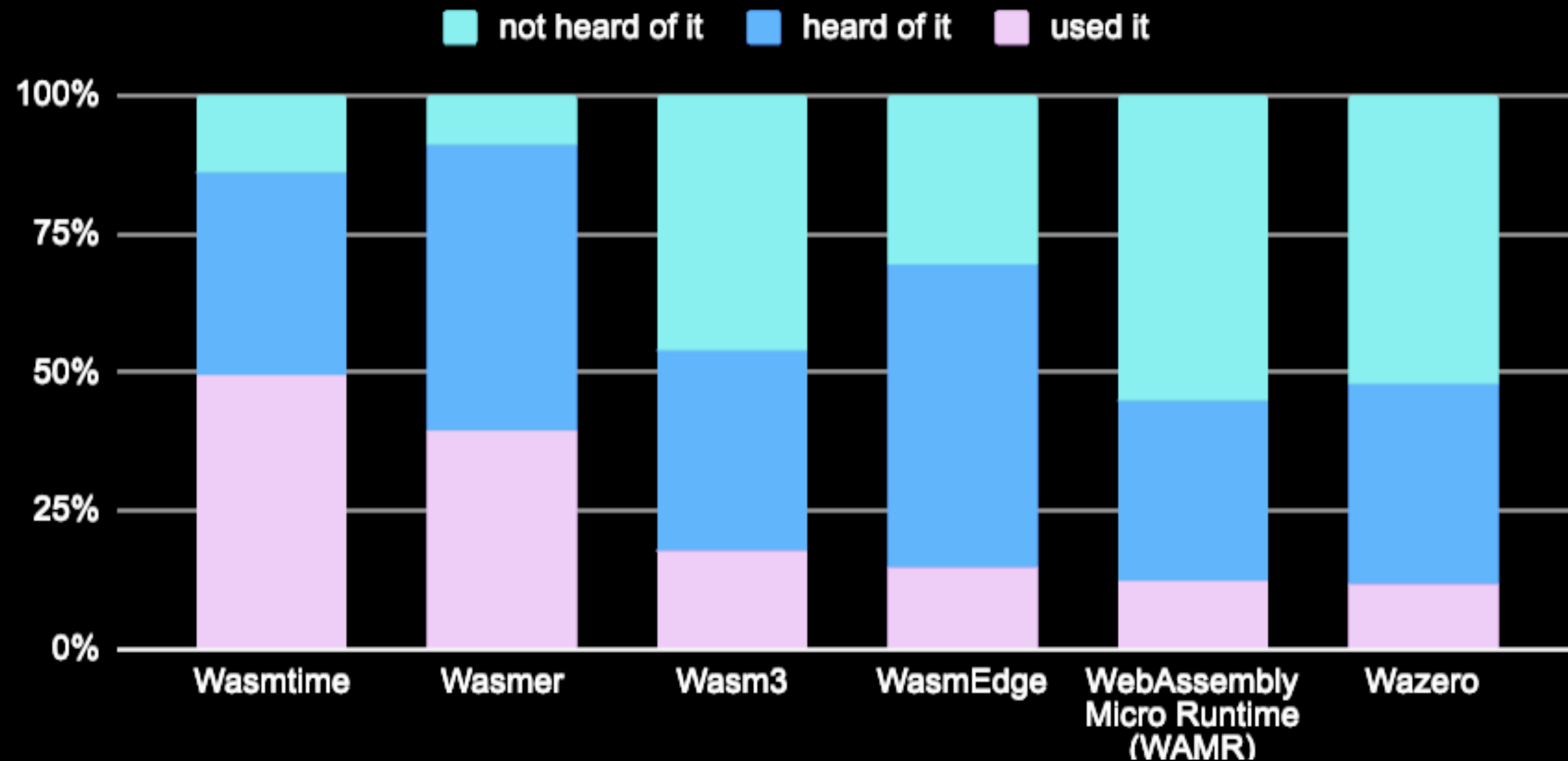


WEBASSEMBLY BEYOND THE BROWSER

- The evolution of WebAssembly from browser-based to non-browser contexts.
- Mirrors evolution of JavaScript to Node.js
- Similarities to Java Virtual Machine.
 - WasmGC is supported in Chrome and Firefox but is still pending in Safari.
 - Kotlin can compile to WASM using WasmGC.

WEBASSEMBLY RUNTIMES

WebAssembly runtime usage



Source: The State of WebAssembly 2023: <https://blog.scottlogic.com/2023/10/18/the-state-of-webassembly-2023.html>

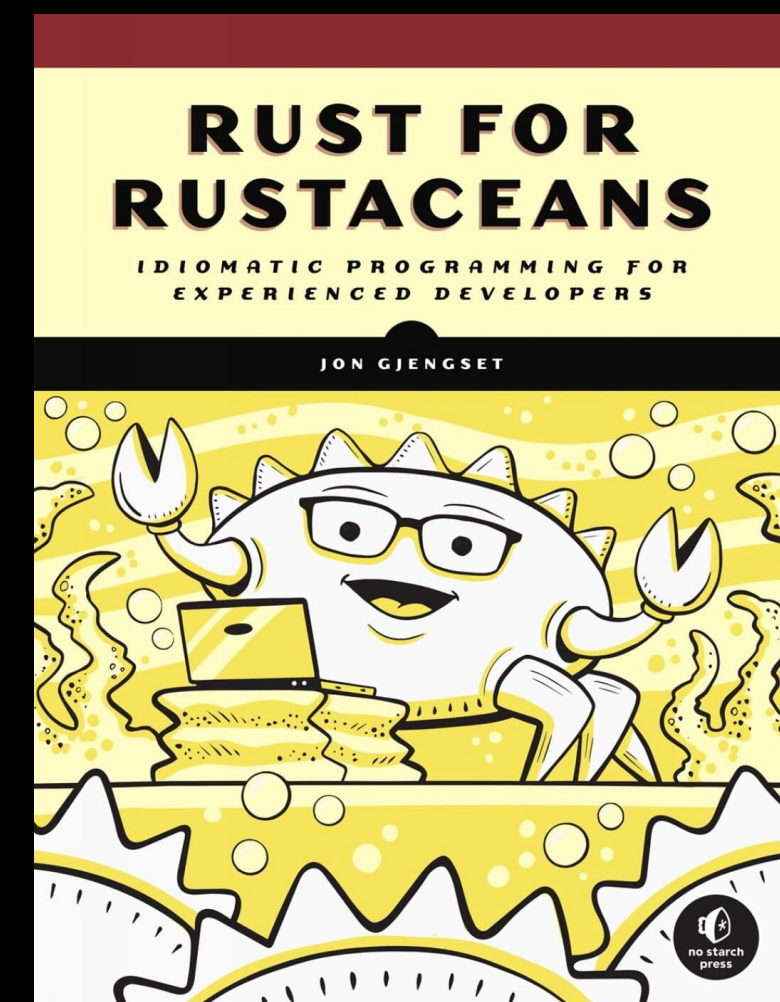
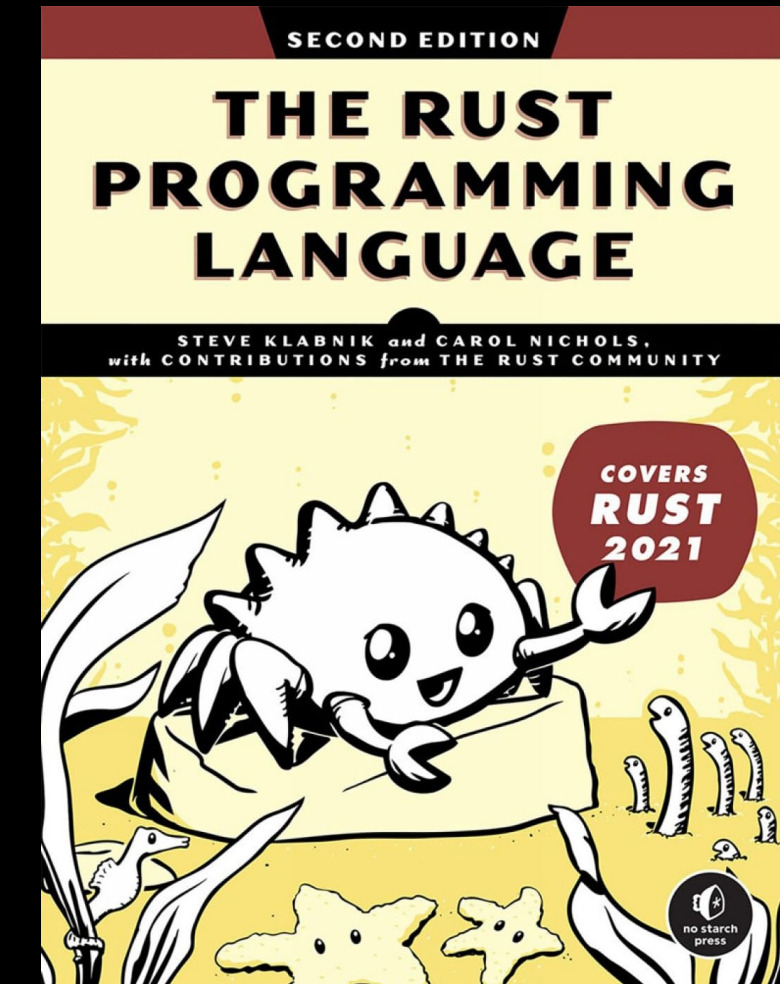


LEARNING MORE ABOUT RUST



LEARNING MORE ABOUT RUST

- "The Book" The Rust Programming Language
 - <https://doc.rust-lang.org/book/>
 - <https://www.amazon.com/Rust-Programming-Language-2nd/dp/1718503105/>
- Let's Get Rusty ("The Book" in videos)
 - <https://www.youtube.com/playlist?list=PLai5B987bZ9CoVR-QEIN9foz4QCJ0H2Y8>
- Rust by Example
 - <https://doc.rust-lang.org/rust-by-example/>
- Rust Playground (Interactive Rust)
 - <https://play.rust-lang.org/>
- Rust for Rustaceans
 - <https://www.amazon.com/Rust-for-Rustaceans/dp/1718501854/>





Thank You

Miami JVM Group (MJUG)

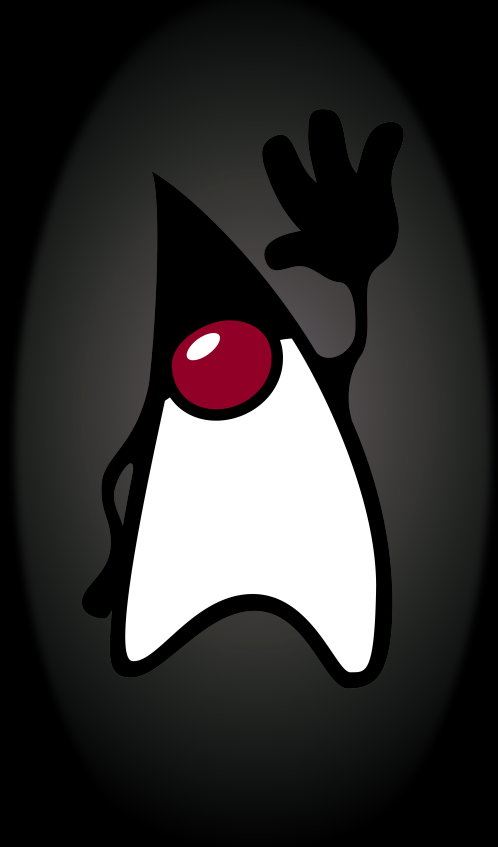
<https://www.meetup.com/miami-java-user-group/>

www.linkedin.com/in/ealvarez



REFERENCES

- 2024 Stack Overflow Developer Survey
- <https://survey.stackoverflow.co/2024/technology#admired-and-desired>
- How Rust went from a side project to the world's most-loved programming language
- <https://www.technologyreview.com/2023/02/14/1067869/rust-worlds-fastest-growing-programming-language/>
- **LLVM** Compiler Infrastructure
- <https://llvm.org/>
- Rustc Platform Support
- <https://doc.rust-lang.org/rustc/platform-support.html>
- Energy Efficiency Across Programming Languages
- <https://sites.google.com/view/energy-efficiency-languages/>
- AWS re:Inforce 2024 - Keynote with Chris Betz
- <https://www.youtube.com/watch?v=skH3Q90llss>



REFERENCES

- First Rust Code in the Windows 11 Kernel
 - <https://www.thurrott.com/windows/windows-11/282995/first-rust-code-shows-up-in-the-windows-11-kernel>
- Rust for Linux
 - https://en.wikipedia.org/wiki/Rust_for_Linux
- In Rust we trust? White House Office urges memory safety
 - <https://stackoverflow.blog/2024/03/04/in-rust-we-trust-white-house-office-urges-memory-safety/>
- The Case for Memory Safe Roadmaps
 - <https://media.defense.gov/2023/Dec/06/2003352724/-1/-1/0/The-Case-for-Memory-Safe-Roadmaps-TLP-clear.pdf>
- Eliminating Memory Safety Vulnerabilities at the Source
 - <https://security.googleblog.com/2024/09/eliminating-memory-safety-vulnerabilities-Android.html>



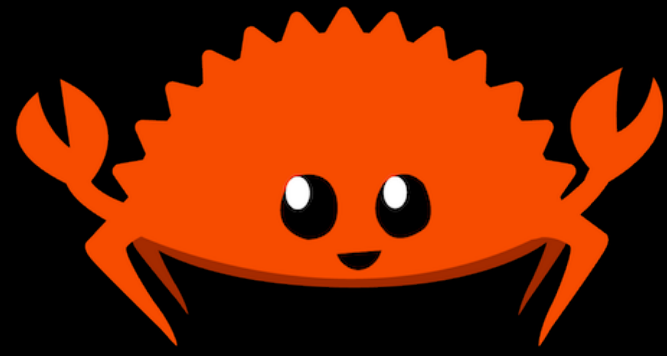
REFERENCES

- Safe C++ proposal
- <https://safecpp.org/draft.html>
- Fil-C
- <https://github.com/pizlonator/llvm-project-deluge/blob/deluge/Manifesto.md>
- TrapC
- <https://vimeo.com/1028578347>
- Enabling hardened libc++ / listedc++
- <https://security.googleblog.com/2024/11/retrofitting-spatial-safety-to-hundreds.html>



REFERENCES

- 2023 JetBrains State of Developer Ecosystem
- <https://www.jetbrains.com/lp/devecosystem-2023/rust/>
- The Rust community's crate registry
- <https://crates.io/>
- 2023 Annual Rust Survey Results
- <https://blog.rust-lang.org/2024/02/19/2023-Rust-Annual-Survey-2023-results.html>
- The State of WebAssembly 2023
- <https://blog.scottlogic.com/2023/10/18/the-state-of-webassembly-2023.html>



REFERENCES

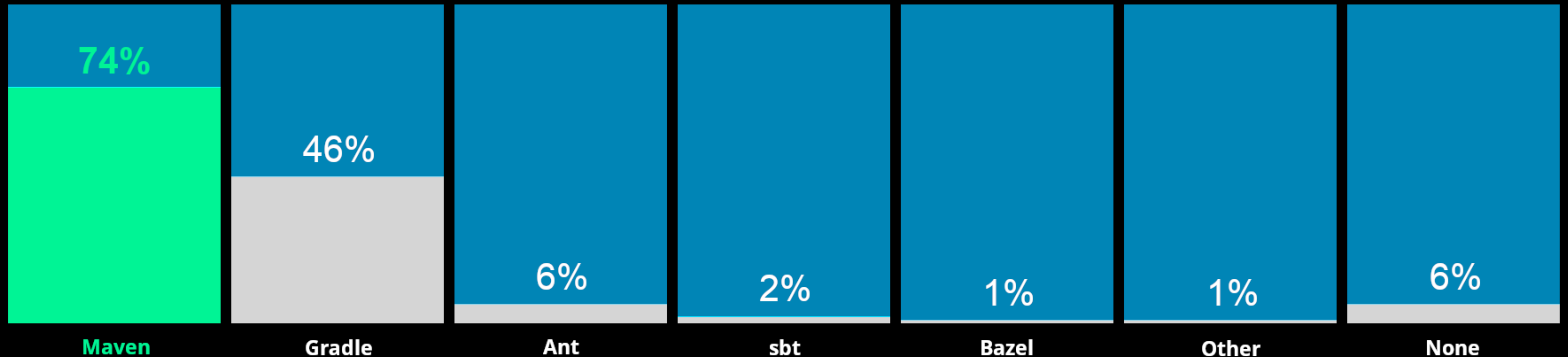
- The Rust Programming Language “The Book”
 - <https://doc.rust-lang.org/book/>
- Let’s Get Rusty (Videos)
 - <https://www.youtube.com/playlist?list=PLai5B987bZ9CoVR-QEIN9foz4QCJ0H2Y8>
- Rust by Example
 - <https://doc.rust-lang.org/rust-by-example/>
- Rust Playground
 - <https://play.rust-lang.org/>
- Rust for Rustaceans
 - <https://www.amazon.com/Rust-Rustaceans-Programming-Experienced-Developers/dp/1718501854/>



ET CETERA

BUILDING JAVA APPLICATIONS

Which build systems do you regularly use?

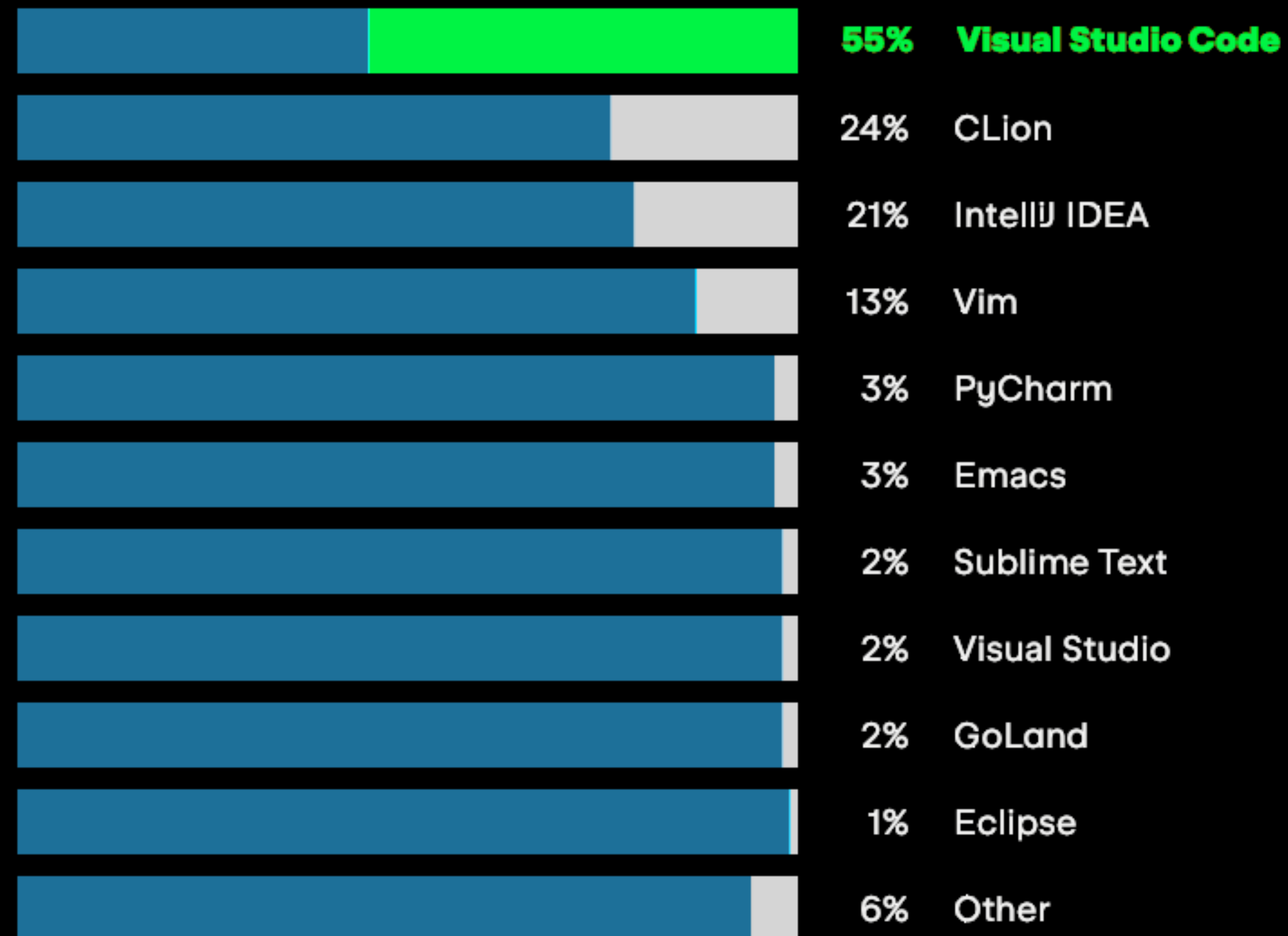


Maven • Gradle • Ant • sbt • Bazel

Source: JetBrains State of Developer Ecosystem 2023 : <https://www.jetbrains.com/lp/devecosystem-2023/java/>

RUST IDE

What IDEs / editors do you use for Rust development?



Source: JetBrains State of Developer Ecosystem 2023 : <https://www.jetbrains.com/lp/devecosystem-2023/rust/>



MICROSOFT BLAZOR

- **C#-Based Web Development:** Enables building interactive web applications using C# and .NET instead of JavaScript.
- **Component-Based Architecture:** Allows developers to create reusable UI components using Razor syntax (HTML + C#).
- **WebAssembly and Server-Side Models:** Supports both Blazor WebAssembly for client-side execution and Blazor Server for server-side execution via SignalR.
- **Seamless Integration:** Works seamlessly with the .NET ecosystem, offering robust support for libraries, tools, and frameworks.
- **Unified Full-Stack Development:** Allows developers to use a single programming language (C#) for both client-side and server-side logic.

Source: Blazor: <https://en.wikipedia.org/wiki/Blazor>